



Assignment 2 (Questions): (Knowing your data, filtering, and sorting)

Supermarket Sales Data Analysis – Filtering, Sorting & Aggregation

Description & Objectives

This assignment evaluates your proficiency in Exploratory Data Analysis using Python Pandas. Working with the `SuperMarket Analysis.csv` dataset (1,000 transactions across three branches), you will demonstrate competency in data ingestion, quality assessment, conditional filtering, multi-criteria sorting, and group-based aggregation. The exercise requires you to extract actionable business insights including revenue patterns by branch/demographic, product performance metrics, and customer behavior indicators—through programmatic data manipulation without modifying source data.

Instructions: Complete the following questions:

Imports

```
In [1]: import pandas as pd
```

Q1. Load the (SuperMarket Analysis.csv) dataset and display the first 5 rows.

```
In [2]: df = pd.read_csv("./SuperMarket Analysis.csv")
df.head()
```

Out[2]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085

Q2. Display the dataset shape (rows and columns)

In [3]: `df.shape`

Out[3]: (1000, 17)

Q3. List all column names

In [4]: `df.columns`

Out[4]: Index(['Invoice ID', 'Branch', 'City', 'Customer type', 'Gender', 'Product line', 'Unit price', 'Quantity', 'Tax 5%', 'Sales', 'Date', 'Time', 'Payment', 'cogs', 'gross margin percentage', 'gross income', 'Rating'], dtype='str')

Q4. Identify categorical and numerical columns.

In [5]: `df.dtypes`

```
Out[5]: Invoice ID          str
        Branch           str
        City             str
        Customer type    str
        Gender           str
        Product line     str
        Unit price       float64
        Quantity         int64
        Tax 5%           float64
        Sales            float64
        Date             str
        Time             str
        Payment          str
        cogs             float64
        gross margin percentage float64
        gross income     float64
        Rating           float64
        dtype: object
```

```
In [6]: categories = ['Branch', 'City', 'Customer type', 'Gender', 'Product line', 'Payment']

for category in categories:
    df[category] = df[category].astype('category')
```

```
In [7]: df.dtypes
```

```
Out[7]: Invoice ID          str
        Branch           category
        City             category
        Customer type    category
        Gender           category
        Product line     category
        Unit price       float64
        Quantity         int64
        Tax 5%           float64
        Sales            float64
        Date             str
        Time             str
        Payment          category
        cogs             float64
        gross margin percentage float64
        gross income     float64
        Rating           float64
        dtype: object
```

Q5. Check for missing values in each column

```
In [8]: df.isna().sum()
```

```
Out[8]: Invoice ID      0
        Branch        0
        City          0
        Customer type  0
        Gender         0
        Product line   0
        Unit price     0
        Quantity       0
        Tax 5%         0
        Sales          0
        Date           0
        Time           0
        Payment        0
        cogs           0
        gross margin percentage  0
        gross income   0
        Rating         0
        dtype: int64
```

Q6. Display the data types of each column

```
In [9]: df.dtypes
```

```
Out[9]: Invoice ID      str
        Branch        category
        City          category
        Customer type  category
        Gender         category
        Product line   category
        Unit price     float64
        Quantity       int64
        Tax 5%         float64
        Sales          float64
        Date           str
        Time           str
        Payment        category
        cogs           float64
        gross margin percentage  float64
        gross income   float64
        Rating         float64
        dtype: object
```

Q7. Show summary statistics for numerical columns (use pandas method)

```
In [10]: numerical_columns = df.select_dtypes(include=['float64', 'int64']).columns
```

```
In [11]: df[numerical_columns].describe()
```

Out[11]:

	Unit price	Quantity	Tax 5%	Sales	cogs	gross margin percentage	
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	10
mean	55.672130	5.510000	15.379369	322.966749	307.58738	4.761905	
std	26.494628	2.923431	11.708825	245.885335	234.17651	0.000000	
min	10.080000	1.000000	0.508500	10.678500	10.17000	4.761905	
25%	32.875000	3.000000	5.924875	124.422375	118.49750	4.761905	
50%	55.230000	5.000000	12.088000	253.848000	241.76000	4.761905	
75%	77.935000	8.000000	22.445250	471.350250	448.90500	4.761905	
max	99.960000	10.000000	49.650000	1042.650000	993.00000	4.761905	



Q8. Filter rows where Sales > 500

In [12]: `df[df['Sales'] > 500].head()`

Out[12]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1411
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2081
5	699-14-3026	Giza	Naypyitaw	Member	Female	Electronic accessories	85.39	7	29.8861
7	315-22-5665	Giza	Naypyitaw	Member	Female	Home and lifestyle	73.56	10	36.7800
14	829-34-3910	Alex	Yangon	Member	Female	Health and beauty	71.38	10	35.6900



Q9. Filter sales in City = "Yangon" and Sales > 200

In [13]: `df[(df['Sales'] > 200) & (df['City'] == 'Yangon')].head()`

Out[13]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415 5
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155 3
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880 4
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085 6
6	355-53-5943	Alex	Yangon	Member	Female	Electronic accessories	68.84	6	20.6520 4



Q10. Sort all orders by Sales in descending order

In [14]: `df.sort_values(by=['Sales'], ascending=False).head()`

Out[14]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
350	860-79-0874	Giza	Naypyitaw	Member	Female	Fashion accessories	99.30	10	49.65
167	687-47-8271	Alex	Yangon	Normal	Male	Fashion accessories	98.98	10	49.49
557	283-26-5248	Giza	Naypyitaw	Member	Female	Food and beverages	98.52	10	49.26
699	751-41-9720	Giza	Naypyitaw	Normal	Male	Home and lifestyle	97.50	10	48.75
996	303-96-2227	Cairo	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.69



Q11. Sort by Date (ascending) and then Time (ascending)

```
In [18]: # df['Date'] = pd.to_datetime(df['Date'], format="%m/%d/%Y")
# df['Time'] = pd.to_datetime(df['Time'], format="%I:%M:%S %p")
df['Date time'] = pd.to_datetime(df['Date'] + ' ' + df['Time'], format="%m/%d/%Y %I:%M:%S %p")
df.sort_values(by=['Date time']).iloc[[0, 1, 2, 997, 998, 999]]
```

Out[18]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
17	765-26-6951	Alex	Yangon	Member	Female	Sports and travel	72.61	6	21.78
970	746-04-1077	Cairo	Mandalay	Member	Female	Food and beverages	84.63	10	42.31
839	271-77-8740	Giza	Naypyitaw	Member	Female	Sports and travel	29.22	6	8.76
473	731-59-7531	Cairo	Mandalay	Member	Male	Health and beauty	72.57	8	29.02
643	131-15-8856	Giza	Naypyitaw	Member	Female	Food and beverages	72.52	8	29.00
963	364-34-2972	Giza	Naypyitaw	Member	Male	Electronic accessories	96.82	3	14.52

Q12. Sort by Unit price and Quantity

```
In [17]: df.sort_values(by=['Unit price', 'Quantity']).head()
```

Out[17]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
944	333-23-2632	Alex	Yangon	Member	Male	Health and beauty	10.08	7	3.5280
572	239-48-4278	Alex	Yangon	Member	Male	Food and beverages	10.13	7	3.5451
784	516-77-6464	Giza	Naypyitaw	Member	Female	Health and beauty	10.16	5	2.5400
822	784-21-9238	Giza	Naypyitaw	Member	Male	Sports and travel	10.17	1	0.5080
881	115-38-7388	Giza	Naypyitaw	Member	Female	Fashion accessories	10.18	8	4.0720



Q13. Calculate the total sales in "Sales" per Branch

In [21]: `df.groupby(by=['Branch'])['Sales'].sum()`

Out[21]:

Branch	Sales
Alex	106200.3705
Cairo	106197.6720
Giza	110568.7065

Name: Sales, dtype: float64

Q14. Calculate average Sales per City

In [23]: `df.groupby(by=['City'])['Sales'].mean()`

Out[23]:

City	Sales
Mandalay	319.872506
Naypyitaw	337.099715
Yangon	312.354031

Name: Sales, dtype: float64

Q15. Find the quantities sold per product line

In [24]: `df.groupby(by=['Product line'])['Quantity'].count()`

```
Out[24]: Product line
Electronic accessories    170
Fashion accessories       178
Food and beverages        174
Health and beauty         152
Home and lifestyle        160
Sports and travel         166
Name: Quantity, dtype: int64
```

Q16. Calculate average gross income per Gender

```
In [25]: df.groupby(by=['Gender'])['gross income'].mean()
```

```
Out[25]: Gender
Female    16.234829
Male      14.240749
Name: gross income, dtype: float64
```

Q17. Count number of sales per Payment method

```
In [27]: df.groupby(by=['Payment'])['Sales'].count()
```

```
Out[27]: Payment
Cash          344
Credit card   311
Ewallet       345
Name: Sales, dtype: int64
```

Q18. Find maximum Sales per Branch.

```
In [28]: df.groupby(by=['Branch'])['Sales'].max()
```

```
Out[28]: Branch
Alex      1039.29
Cairo     1022.49
Giza      1042.65
Name: Sales, dtype: float64
```

Q19. Find minimum Unit price per Product line.

```
In [29]: df.groupby(by=['Product line'])['Unit price'].min()
```

```
Out[29]: Product line
Electronic accessories    10.56
Fashion accessories       10.18
Food and beverages        10.13
Health and beauty         10.08
Home and lifestyle        10.53
Sports and travel         10.17
Name: Unit price, dtype: float64
```

Q20. Find the sum of gross income per Product line and Branch.

```
In [30]: df.groupby(by=['Product line', 'Branch'])['gross income'].sum()
```

```
Out[30]:
```

Product line	Branch	
Electronic accessories	Alex	872.2435
	Cairo	811.9735
	Giza	903.2845
Fashion accessories	Alex	777.7385
	Cairo	781.5865
	Giza	1026.6700
Food and beverages	Alex	817.2905
	Cairo	724.5185
	Giza	1131.7550
Health and beauty	Alex	599.8930
	Cairo	951.4600
	Giza	791.2060
Home and lifestyle	Alex	1067.4855
	Cairo	835.6745
	Giza	661.6930
Sports and travel	Alex	922.5095
	Cairo	951.8190
	Giza	750.5680

Name: gross income, dtype: float64

Q21. What is the total quantities sold in Product line: "Electronic accessories"?

```
In [33]: (df[df['Product line'] == 'Electronic accessories'])['Quantity'].sum()
```

```
Out[33]: np.int64(971)
```

Q22. What is the average Sales for female customers?

```
In [35]: (df[df['Gender'] == 'Female'])['Sales'].mean()
```

```
Out[35]: np.float64(340.9314141856392)
```

Q23. What is the most expensive Unit price among Customer type members only?

```
In [37]: (df[df['Customer type'] == 'Member'])['Unit price'].max()
```

```
Out[37]: np.float64(99.96)
```

Q24. How many orders with Rating >= 9 ?

```
In [39]: (df[df['Rating'] >= 9])['Invoice ID'].count()
```

```
Out[39]: np.int64(166)
```

Q25. What is the total Sales for Payment " Credit card " in Branch C?

```
In [43]: (df[(df['Payment'] == 'Credit card') & (df['Branch'] == 'Cairo')])['Sales'].sum()
```

```
Out[43]: np.float64(37344.8565)
```